

Table 2

Low-code platform	Description	Type
Mendix	This is designed to accelerate enterprise app delivery across the entire application development life cycle, from ideation to development, deployment, and maintenance on cloud or on-premise.	Proprietary
OutSystems	This platform provides tools to develop, deploy and manage omni-channel enterprise applications. It addresses the full spectrum of enterprise use cases for mobile, Web and core systems.	Proprietary
Appian	An enterprise grade platform that combines the key capabilities needed to get work done faster using AI, RPA, decision rules, and workflow on a single low-code platform.	Proprietary
Budibase	Helps in building business applications. Supports multiple external data sources, and comes with prebuilt layouts, user auth, and a data provider component. Supports JavaScript for integrations.	Open source
WordPress	Powers more than 41 per cent of the Web -- from simple blogs to enterprise web-sites. With over 54,000 plugins, the level of customisation without writing code is incredible.	Open source
Node-RED	Helps to build event-driven IoT applications. A programming tool for wiring together hardware devices, APIs, and online services.	Open source

integration options (custom, data, UI, infra) to support building next-gen low-code products with multi-cloud deployment options. They are able to scale and handle thousands of users and millions of data sets.

Pros and cons of low-code platforms

The primary advantage of low-code software development is speed (months to weeks). On an average, there is six to ten times productivity improvement using a low-code platform over traditional hand coding approaches to software development.

Figure 2 lists some of the key benefits of low-code platforms.

- **Better software, lower maintenance:** Low-code platforms standardise the development approach and reduce the complexity as well as the error rate of the source code.
- **Cross-team collaboration:** Team members with different skills and capabilities can collaborate to realise the final product faster.
- **Enable Agile software development:** They offer the team members a consistent product that begins as a single screen, and grows from sprint to sprint as the full product takes shape.
- **Faster legacy app modernisation:** Enable faster UI generation based on the existing data model, reuse of

the logic from legacy databases and smart transfer of existing screens/persistence layers.

- **Low risk and high RoI:** Due to shorter development cycles, the risk of undertaking a new project is low, and there is a good chance of getting high returns.
 - **Scaling through multiple components:** Allow use of a common platform to develop multiple services.
 - **Easy maintenance:** The software can be easily updated, fixed and changed according to customer requirements.
- In the last few years, low-code platforms have evolved a lot in terms of functionality and applicability. However, they still have some limitations when being used fully for any generic product development, some of which are listed below.

- **Low-level product development:** Cannot be used for building products like in-memory grids, Big Data processing algorithms, image recognition, etc.
- **Custom architectures and services:** Have limited use in enterprise software with unique architecture, microservices, custom back-ends, unique enterprise service bus, etc.
- **Source code control:** These platforms totally lose control over the code base; it's difficult to debug and handle edge conditions where the tool does not do the right thing automatically.
- **Limited integration:** Custom integration with legacy systems needs some significant coding.
- **Security and reliability:** These platforms are vulnerable to security breaches, because if the low-code platform gets hacked, it can immediately make the product built on it also vulnerable.
- **Customisation:** Custom CSS, custom integration, advanced client-side functionality, etc, will require a good amount of coding.